

Problem Set 3 - Optimization, and Numerical Integration and Differentiation

Fred Milhome

2026-03-26

To reproduce the results in this report, get the replication files in [my GitHub repository pset3 folder](#) and follow the README.md. You're supposed to simply render this report if you did everything right.

Below we just set up the environment.

```
include("code/setup_env.jl")

# External packages
using Random, PrettyTables, Printf, DataFrames, Optim, NLOpt,
    ↪ ForwardDiff, FastGaussQuadrature, Plots
mkpath("figures")

# My functions and types for this problem set are packed in this module
include("code/Pset3.jl")
using .Pset3

# Set seed; used in random optimization algorithms and Monte Carlo
    ↪ integration
Random.seed!(201010)
```

Part 1: Optimization

For this part, the chosen optimization algorithms are Nelder-Mead, Gradient Descent, CRS, and Simulated Annealing.

```
myf = x -> x * sin(5x)

lb_opt = [0.0]
ub_opt = [10.0]
```

```

optimizers = [NelderMeadI(x0=0),
              GradientDescentI(x0=0),
              CRSI(lb=lb_opt, ub=ub_opt, N_args=1, pop = 300, maxeval =
                ↪ 150),
              SimAnnI(lb=0.0, ub=10.0, x0=0, temp_schedule_par = 0.5,
                ↪ maxiter=250)]

names      = ["Nelder-Mead", "Gradient Descent", "CRS", "Simulated
  ↪ Annealing"]
results = [numoptimize(myf, opt) for opt in optimizers]
# Table snippet omitted

```

Algorithm	Minimizer	f^*	Function Evaluations
Nelder-Mead	0.000000	0.000000	13
Gradient Descent	0.000000	0.000000	1
CRS	9.688598	-9.383328	150
Simulated Annealing	9.743027	-9.740990	250

If we started with $x_0 = 10$ for Nelder-Mead and Gradient Descent, they would have managed to find the global minimum, as we can see below (code snippet omitted).

Algorithm	Minimizer	f^*	Function Evaluations
Nelder-Mead	1228.562500	-1033.043783	20
Gradient Descent	5.975717	-5.972373	18
CRS	9.844076	-8.514900	150
Simulated Annealing	9.743158	-9.740988	250

Part 2: Minimizing the quadratic difference

```

function tgt_f(x::AbstractVector{<:Real}, ::AbstractVector{<:Real})
    return [1] * x[1] + [2] * exp(-x[2]*x[2]) + [3] *
      ↪ log(1+abs(x[2])) + [4]*(x[1]^x[2])
end

# We'll use trace to keep track of the optimization path and answer item
  ↪ d
trace = DataFrame(eval = Int[], theta = Vector{Float64}[], value =
  ↪ Float64[])
k = Ref{0} # To keep track of the number of evaluations

function g(::AbstractVector{<:Real})
    known_values = [(1.0, 1.0) => 43.614,
                    (2.0, 4.0) => 563.694,
                    (-1.0, 2.0) => 43.23,
                    (2.0, -2.0) => 23.13
                    ]
    result = sum((tgt_f(collect(x), ) - y)^2.0 for (x, y) in
  ↪ known_values)

```

```

    # Record the evaluation in the trace
    push!(trace, (eval = k[], theta = collect(), value = result))
    k[] += 1
    return result
end

val_b = @sprintf("%.6f", g([0.0, 0.0, 0.0, 0.0]))
display("text/latex", "Item b: \\\$g\\\$\\theta = [0, 0, 0, 0]) = \$val_b\\$.")

```

Item b: $g(\theta = [0, 0, 0, 0]) = 322056.936432$.

To minimize the g function, I chose the Simulated Annealing algorithm in Optim without bounds, which allows us to explore different regions of the parameter space by controlling the temperature function schedule parameter, and requires us to set the variance of the new draws. The algorithm is run for 10,000 iterations, which seem enough to minimize the function as we'll see below. We begin from the initial guess of $\theta = [0, 0, 0, 0]$ as I don't know anything yet.

```

opt = SimAnnNBI(x0=[0.0, 0.0, 0.0, 0.0], step_size=0.01, maxiter=100_000)
result = numoptimize(g, opt)
fmin_str = @sprintf("%.6f", result.f_min)
min_str = join([@sprintf("%.2f", x) for x in result.minimizer], ", ")
display("text/latex", "Item c: Optimal \\\$g\\\$ is \\\$fmin_str\\$, with
↪ minimizer \\\$min_str\\$.")

```

Item c: Optimal g is 0.000215, with minimizer $[1.80, 2.38, 10.00, 34.00]$.

Let's analyze the trace of the optimization to see how the algorithm converged and if 100,000 iterations were enough.

```

sum_trace = trace_summary(trace; block=100) # see Utils.jl

# Plot snippet omitted

```

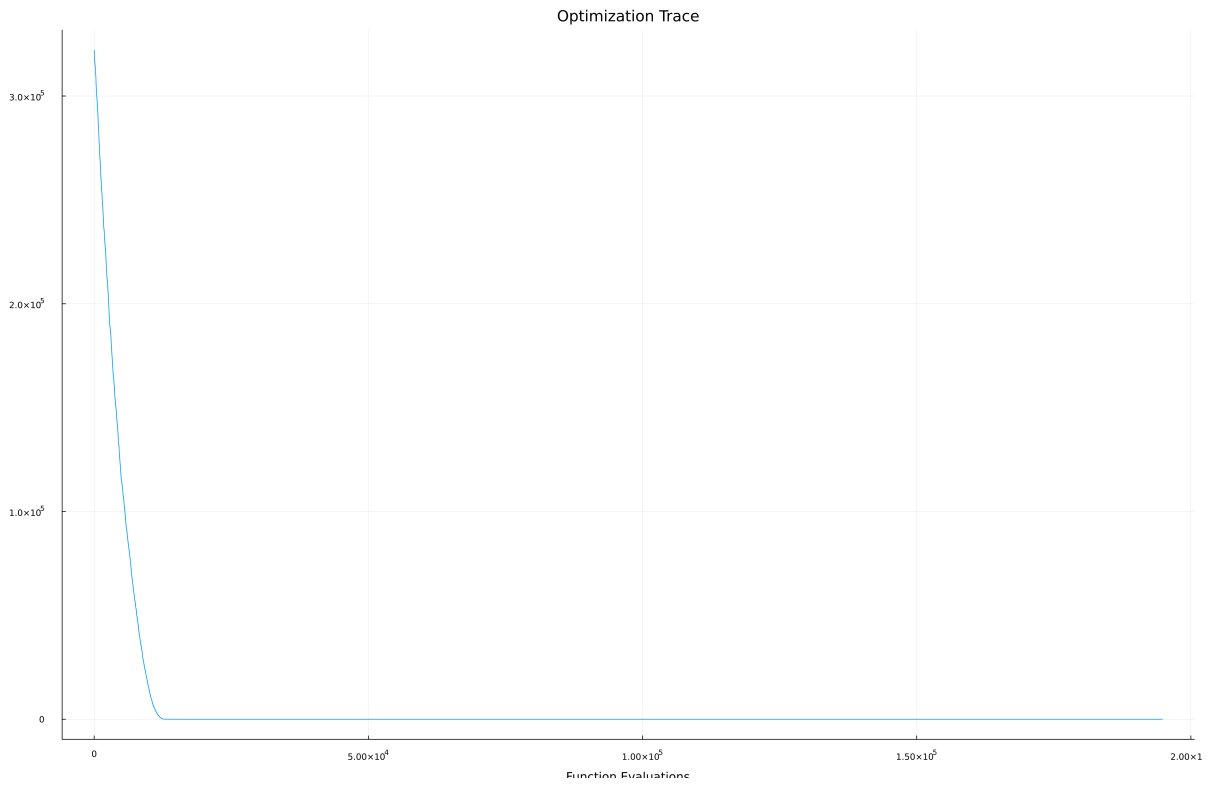


Figure 1: Optimization Trace

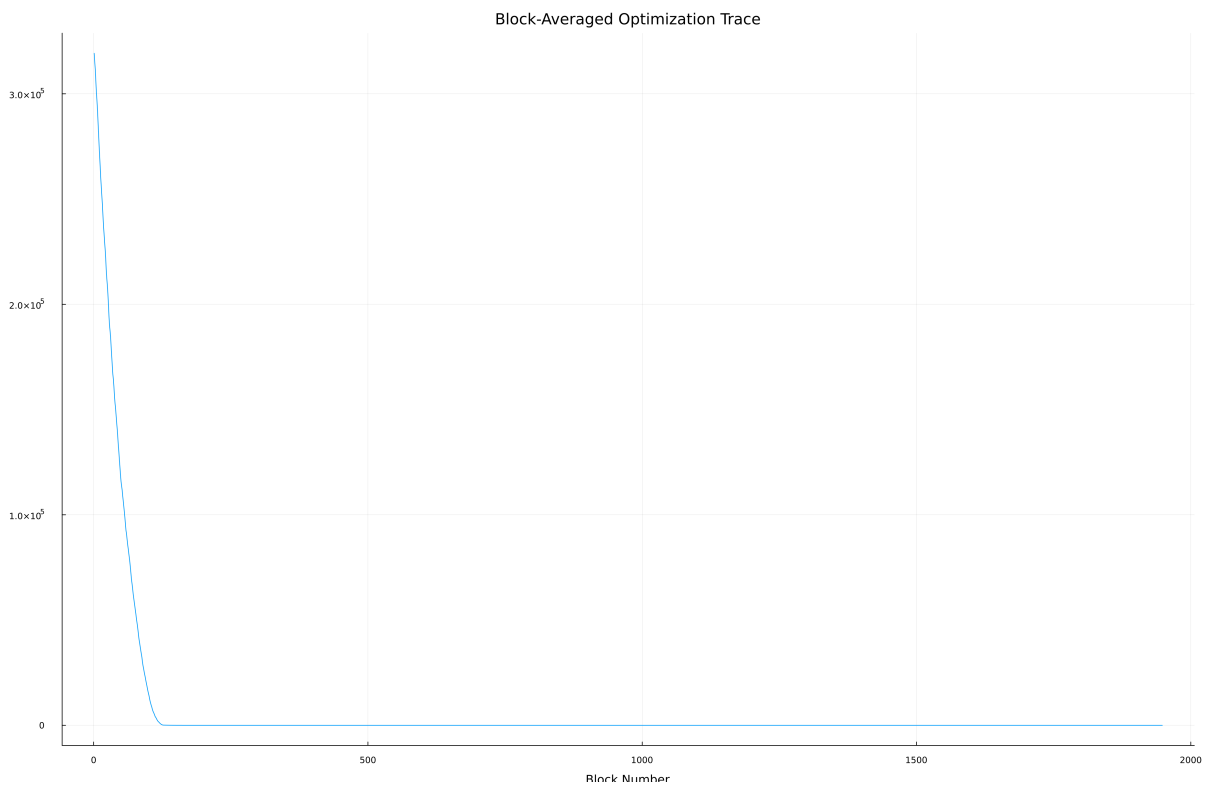


Figure 2: Block-Averaged Optimization Trace

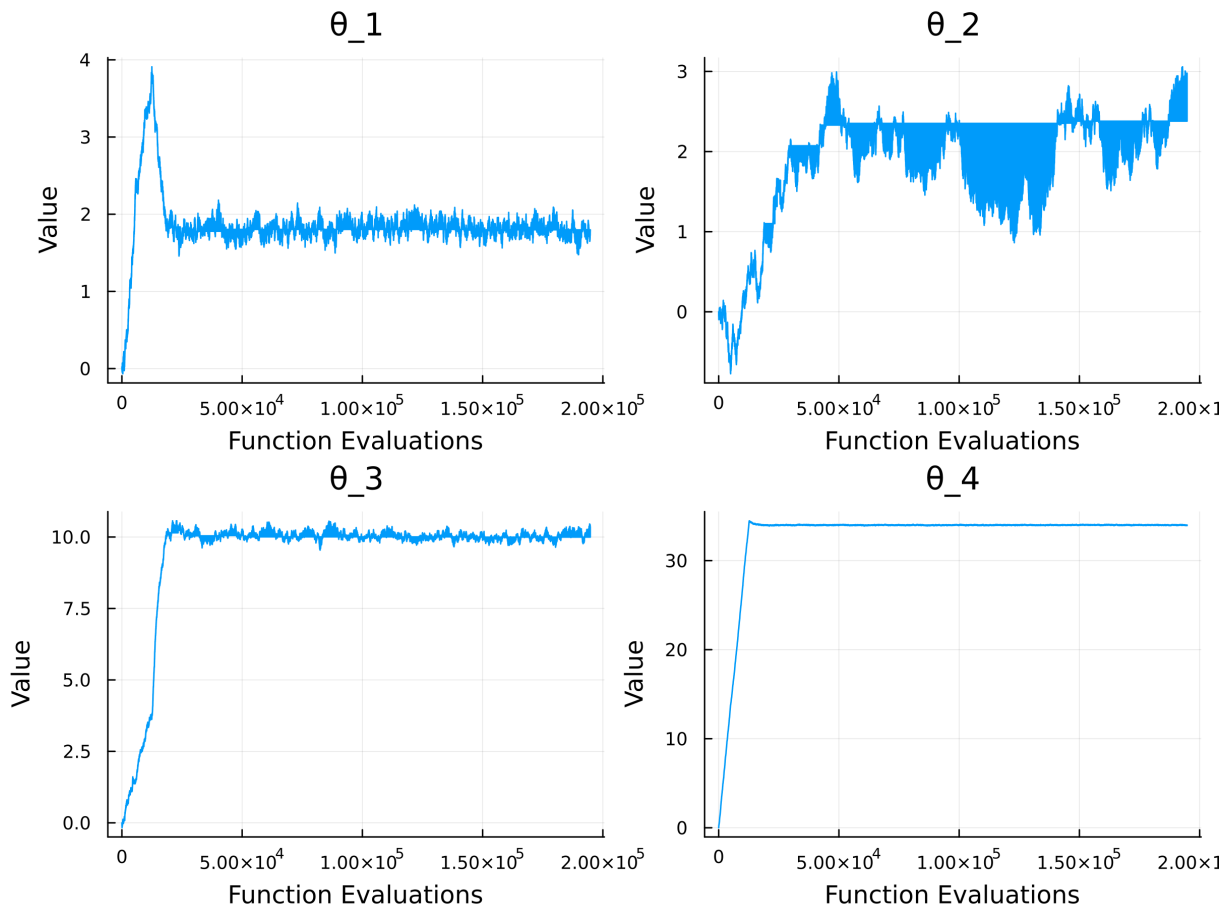


Figure 3: Parameter Traces

It seems enough, but I was puzzled by why the parameters converged so differently. We can think that as we're drawing points from a normal, parameters that cause large changes in g (e.g., θ_3) will have more volatile traces and frequently exploit some flat regions, as several draws get rejected.

Part 3: Expectations

Gauss-Hermite Quadrature

For Gauss-Hermite, we *could* do a 2D quadrature, but the order-statistic identity for the maximum of k i.i.d. standard normal random variables allows us to reduce the problem to a 1D integral, which is much more efficient to compute.

```
using Distributions: cdf, Normal

# Order-statistic identity:  $E[\max(X,Y)] = 2 \cdot E[X \cdot \Phi(X)]$  for  $X, Y \sim N(0,1)$ 
#   ↪ i.i.d.
Φ(x) = cdf(Normal(), x)
u_1d(x) = 2 * x * Φ(x)

result_gh = numexpectation(u_1d, GaussHermiteI(n=10))
gh_str = @sprintf("%.6f", result_gh)
display("text/latex", "The expected utility is \$$gh_str\$ using
  ↪ Gauss-Hermite quadrature with 10 nodes.")
```

The expected utility is 0.564203 using Gauss-Hermite quadrature with 10 nodes.

Monte Carlo Integration

```
# Generate draws
num_draws = 100_000
sample = [randn(num_draws) randn(num_draws)]

# Honestly I just did it like this for completeness, because my method
↳ just takes an average... but a more flexible implementation could
↳ take a pdf and weight the draws if necessary.
result_mc = numexpectation(u, sample, MonteCarloI())
mc_str = @sprintf("%.6f", result_mc)
display("text/latex", "The expected utility is \$$mc_str\$ using Monte
↳ Carlo integration with $num_draws draws.")
```

The expected utility is 0.562077 using Monte Carlo integration with 100000 draws.

Part 4: Integration

```
n_list = [3, 5, 10, 15, 20]

functions = ["x"          => x -> x,
            "x\\sin(x)"   => x -> x * sin(x),
            "\\sqrt{1-x^2}" => x -> sqrt(1-x^2)]

analytical_integrals = [x -> x^2/2,
                       x -> sin(x) - x * cos(x),
                       x -> (1/2) * (x * sqrt(1-x^2) + asin(x))
                       ]

for (name, f) in functions
    results = [numintegrate(f, 0, 1, TrapezoidalI(n=n)) for n in n_list]
    io = IOBuffer()
    pretty_table(io, DataFrame(n = n_list, Integral = results);
        column_labels = [LatexCell("\$n\$"), LatexCell("Value")],
        formatters     = [(v, i, j) -> j == 2 ? @sprintf("%.6f", v) : v],
        backend        = :latex)
    display("text/latex", "\$f(x) = \$name\$\\n\\n" * String(take!(io)))
end
```

$$f(x) = x$$

n	Value
3	0.500000
5	0.500000
10	0.500000
15	0.500000
20	0.500000

$$f(x) = x \sin(x)$$

n	Value
3	0.314016
5	0.305781
10	0.302321
15	0.301681
20	0.301457

$$f(x) = \sqrt{1 - x^2}$$

n	Value
3	0.729388
5	0.759262
10	0.776130
15	0.780348
20	0.782116

Part 5: Differentiation

```

hstep_list = [0.001, 0.005, 0.01, 0.05]
order_list = [2, 4, 6]
functions = ["x"           => x -> x*x,
             "\\log(x)"    => x -> log(x),
             "x \\sin(x)" => x -> x * sin(x)
            ]

analytical_derivatives = [x -> 2x,
                          x -> 1/x,
                          x -> sin(x) + x*cos(x)
                         ]

eval_x = [5.0, 10.0, 1.0]

rows = [(ord, h) for ord in order_list for h in hstep_list]
n      = length(rows)

data = Matrix{Any}(undef, n, 2 + 2*length(functions))
for (i, (ord, h)) in enumerate(rows)
    data[i, 1] = ord
    data[i, 2] = h
    for (j, ((_, f), f)) in enumerate(zip(functions,
        ↪ analytical_derivatives))
        v = numdifferentiation(x -> f(x[1]), [eval_x[j]], CentralFDI(h=h,
        ↪ coefs=ord))

```

```

    data[i, 2 + 2*(j-1) + 1] = v
    data[i, 2 + 2*(j-1) + 2] = abs(v - f(eval_x[j]))
end
end
# Table snippet omitted

```

<i>Order</i>	<i>h</i>	x <i>f</i>	x <i>Error</i>	$\log(x)$ <i>f</i>	$\log(x)$ <i>Error</i>	$x \sin(x)$ <i>f</i>	$x \sin(x)$ <i>Error</i>
2	0.0010	10.000000	3.34e-12	0.100000	3.33e-10	1.381773	5.11e-07
2	0.0050	10.000000	2.13e-13	0.100000	8.33e-09	1.381761	1.28e-05
2	0.0100	10.000000	2.13e-13	0.100000	3.33e-08	1.381722	5.11e-05
2	0.0500	10.000000	3.55e-14	0.100001	8.33e-07	1.380497	1.28e-03
4	0.0010	10.000000	5.41e-12	0.100000	1.59e-13	1.381773	2.82e-13
4	0.0050	10.000000	9.41e-14	0.100000	3.32e-14	1.381773	9.89e-11
4	0.0100	10.000000	2.72e-13	0.100000	9.61e-14	1.381773	1.58e-09
4	0.0500	10.000000	1.24e-14	0.100000	5.00e-11	1.381772	9.89e-07
6	0.0010	10.000000	5.00e-12	0.100000	4.08e-14	1.381773	1.81e-13
6	0.0050	10.000000	6.04e-14	0.100000	6.28e-14	1.381773	3.24e-14
6	0.0100	10.000000	7.82e-14	0.100000	2.14e-15	1.381773	4.15e-14
6	0.0500	10.000000	8.88e-15	0.100000	9.85e-15	1.381773	7.17e-10

We can see above that the error from the analytical derivative generally decreases as we increase the order of the method and decrease the step size, which is what we would expect. However, for very small step sizes, the error starts to increase again due to numerical issues.